

Deployment Task Guide

Employee Leave Management System

Django Backend · React Frontend · CI/CD · Domain · SSL

A task-by-task checklist — no guesswork, just steps to follow

Platform Options at a Glance

Layer	Recommended Platform	Free Tier?	Notes
Backend API	Railway (primary rec.)	Yes	Django + PostgreSQL in one place, easy env vars
Backend API	Render	Yes	Similar to Railway, auto-deploy from GitHub
Backend API	AWS Elastic Beanstalk	12 months	More setup needed, good for learning AWS
Backend API	Azure App Service	Yes (F1)	Use if org already on Azure
Frontend	Vercel (primary rec.)	Yes	Best for React — auto-deploy, CDN, free HTTPS
Frontend	Netlify	Yes	Same as Vercel, either works
Database	Railway PostgreSQL	Yes	Bundled with backend on Railway
Database	Supabase	Yes	If using separate DB service
Domain	Namecheap / GoDaddy / Porkbun	No (\$10/yr)	Buy .com or .in domain, point DNS to platforms
CI/CD	GitHub Actions	Free	Works with all platforms above

✓ Railway + Vercel = fastest path. Both free tier, both auto-deploy from GitHub. Recommended for internship.

■ AWS / Azure work fine but require more configuration steps. Use only if project specifically requires cloud provider experience.

1. Repository Setup

Must do this first — everything else depends on it

#	Task	Where	Do ne
1	Create GitHub account if not already done	github.com	■
2	Create new repository named leave-management — set to Private	GitHub → New Repository	■
3	Initialize repo with a README file	GitHub	■
4	Create two folders in repo root: backend/ for Django, frontend/ for React	Your local machine	■
5	Create .gitignore file — add: .env, __pycache__, node_modules, venv, *.pyc, build/	Repo root	■
6	Push all project code to GitHub main branch	Git → GitHub	■
7	Confirm both backend/ and frontend/ folders visible on GitHub	GitHub repo page	■

2. Environment Variables

Define these before touching any platform

Each platform has its own place to enter env vars. Never put these in code files.

Variable Name	What It Is	Set Where
DJANGO_SECRET_KEY	Long random string, 50+ chars	Backend platform env vars
DEBUG	Must be False in production	Backend platform env vars
DATABASE_URL	Full DB connection string	Backend platform — auto-set by Railway
ALLOWED_HOSTS	Your domain + platform domain	Backend platform env vars
CORS_ALLOWED_ORIGINS	Frontend URL (Vercel URL)	Backend platform env vars
REACT_APP_API_URL	Backend API URL + /api	Vercel / Netlify env vars
SSH_PRIVATE_KEY	Deploy key for VPS (if using VPS)	GitHub Secrets only

■ Generate DJANGO_SECRET_KEY at: djecrety.ir or miniwebtool.com/django-secret-key-generator

3. Backend Deploy — Railway (Recommended)

Django API + PostgreSQL together, free tier

#	Task	Where	Do ne
1	Go to railway.app → Sign up with GitHub account	railway.app	■
2	Click New Project → Deploy from GitHub repo	Railway dashboard	■
3	Select your leave-management repository	Railway	■
4	Set Root Directory to backend in Railway service settings	Railway → Service → Settings	■
5	Add PostgreSQL plugin to project — Railway auto-creates DATABASE_URL	Railway → New Plugin → PostgreSQL	■
6	Go to Variables tab → add all backend env vars from Section 2	Railway → Variables	■
7	Add a Procfile in backend/ folder with content: <i>web: gunicorn leaveapp.wsgi</i>	backend/Procfile file	■
8	Add runtime.txt in backend/ with content: <i>python-3.11.0</i>	backend/runtime.txt	■
9	Ensure requirements.txt exists in backend/ with all packages listed	backend/requirements.txt	■
10	Trigger first deploy — Railway builds automatically	Railway → Deploy	■
11	After deploy succeeds → go to Settings → Generate Domain — copy this URL	Railway service	■
12	Run Django migrations: Railway → service → Shell tab → type: <i>python manage.py migrate</i>	Railway Shell	■
13	Create superuser (HR Admin): Railway Shell → <i>python manage.py createsuperuser</i>	Railway Shell	■
14	Test: open the Railway domain URL + /api/ in browser — should return JSON response	Browser	■

✓ Railway auto-redeploys every time you push to GitHub main branch. No extra setup needed.

4. Backend Deploy — Render (Alternative)

Skip if using Railway

#	Task	Where	Do ne
1	Go to render.com → Sign up with GitHub	render.com	■
2	Click New → Web Service → connect your GitHub repo	Render dashboard	■

3	Set Root Directory to backend	Render service config	■
4	Set Build Command to: <code>pip install -r requirements.txt</code>	Render → Build Command	■
5	Set Start Command to: <code>gunicorn leaveapp.wsgi</code>	Render → Start Command	■
6	Add PostgreSQL database via Render → New → PostgreSQL	Render dashboard	■
7	Copy internal DB URL from Render PostgreSQL → paste as DATABASE_URL env var	Render → Environment	■
8	Add all other env vars in Environment tab	Render → Environment	■
9	Deploy → wait for build to finish → copy your Render service URL	Render	■
10	Run migrations via Render Shell tab	Render → Shell	■

5. Backend Deploy — AWS / Azure

Only if cloud provider experience required

AWS Elastic Beanstalk Path

#	Task	Where	Do ne
1	Create AWS account at aws.amazon.com	AWS	■
2	Install AWS CLI and EB CLI on your machine	Your machine	■
3	Create RDS PostgreSQL instance in AWS console → note endpoint + credentials	AWS RDS	■
4	Create Elastic Beanstalk app → Platform: Python → upload backend/ as zip	AWS EB Console	■
5	Set all env vars in EB → Configuration → Software → Environment Properties	AWS EB	■
6	Set DATABASE_URL to point to RDS instance endpoint	AWS EB env vars	■
7	SSH into EB instance → run migrations manually first time	EB SSH	■
8	Note your EB environment URL (e.g. yourapp.elasticbeanstalk.com)	AWS EB dashboard	■

Azure App Service Path

#	Task	Where	Do ne
1	Create Azure account at azure.microsoft.com	Azure portal	■
2	Create Azure Database for PostgreSQL → Flexible Server → note credentials	Azure → Create resource	■
3	Create App Service → Runtime: Python 3.11 → Linux	Azure portal	■
4	Go to App Service → Configuration → add all env vars	Azure App Service	■
5	Go to Deployment Center → connect GitHub repo → set folder to backend/	Azure → Deployment Center	■
6	Set Startup Command to: <code>gunicorn leaveapp.wsgi</code>	Azure → Configuration → General Settings	■
7	Trigger deploy → wait → open App Service URL to verify	Azure	■
8	Run migrations via Azure → App Service → SSH tab or Kudu console	Azure SSH / Kudu	■

■ AWS / Azure free tiers have expiry or credit limits. Railway and Render are simpler for an internship project.

6. Frontend Deploy — Vercel (Recommended)

React SPA — free, CDN, HTTPS automatic

#	Task	Where	Done
1	Go to vercel.com → Sign up with GitHub	vercel.com	■
2	Click Add New Project → import your leave-management repo	Vercel dashboard	■
3	Set Root Directory to frontend	Vercel project config	■
4	Framework Preset: select Create React App	Vercel project config	■
5	Go to Environment Variables section → add REACT_APP_API_URL = your Railway/Render URL + /api	Vercel → Env Variables	■
6	Click Deploy → wait 2-3 minutes → Vercel gives you a free URL (yourapp.vercel.app)	Vercel	■
7	Open the Vercel URL in browser → login page must appear	Browser	■
8	Test login → confirm API calls reach backend (check browser DevTools → Network tab)	Browser DevTools	■
9	Go to backend env vars → update CORS_ALLOWED_ORIGINS to include your Vercel URL	Railway / Render → Variables	■
10	Test login again after CORS update — should work without errors	Browser	■

✓ Vercel auto-redeploys frontend on every push to main. No extra config needed after initial setup.

Frontend on Netlify (Alternative to Vercel)

#	Task	Where	Done
1	Go to netlify.com → Sign up with GitHub	netlify.com	■
2	Click Add new site → Import from Git → select repo	Netlify	■
3	Set Base Directory: frontend , Build Command: npm run build , Publish Directory: frontend/build	Netlify deploy settings	■
4	Add REACT_APP_API_URL in Site settings → Environment variables	Netlify	■
5	Deploy → copy Netlify URL → update CORS on backend	Netlify + Railway/Render	■

7. Custom Domain Setup

Optional but makes project look professional

Without custom domain: app runs on `yourapp.vercel.app` and `yourapi.railway.app` — totally fine for internship. With domain: looks like a real product.

Step A — Buy a Domain

#	Task	Where	Do ne
1	Go to namecheap.com or porkbun.com (cheapest) or godaddy.com	<i>Browser</i>	■
2	Search for your domain name (e.g. <code>leaveapp.in</code> or <code>yourname-leave.com</code>)	<i>Domain registrar</i>	■
3	Buy the domain — cost ~\$700–\$1200/year for <code>.in</code> , ~\$10/year for <code>.com</code>	<i>Domain registrar checkout</i>	■

Step B — Connect Domain to Frontend (Vercel)

#	Task	Where	Do ne
1	Go to Vercel → your project → Settings → Domains	<i>Vercel</i>	■
2	Type your domain (e.g. <code>leaveapp.in</code>) → click Add	<i>Vercel</i>	■
3	Vercel shows a CNAME record value — copy it	<i>Vercel</i>	■
4	Go to your domain registrar → DNS settings → add CNAME record with the value Vercel gave	<i>Namecheap / GoDaddy DNS panel</i>	■
5	Wait 10–30 minutes for DNS propagation	<i>Wait</i>	■
6	Vercel auto-issues SSL certificate — HTTPS works automatically	<i>Vercel (auto)</i>	■
7	Visit your domain in browser → React app should load	<i>Browser</i>	■

Step C — Connect Subdomain to Backend API (Railway)

#	Task	Where	Do ne
1	Decide on API subdomain, e.g. api.leaveapp.in	<i>Your decision</i>	■
2	Go to Railway → your service → Settings → Custom Domain → add <code>api.leaveapp.in</code>	<i>Railway</i>	■
3	Railway shows a CNAME value — copy it	<i>Railway</i>	■
4	Go to domain registrar DNS → add new CNAME record: name= <code>api</code> , value= <code>Railway CNAME</code>	<i>Domain registrar DNS</i>	■

5	Wait 10–30 minutes → Railway auto-issues SSL	<i>Wait</i>	■
6	Update REACT_APP_API_URL in Vercel to: https://api.leaveapp.in/api	<i>Vercel → Env Vars</i>	■
7	Update ALLOWED_HOSTS in Railway to include api.leaveapp.in	<i>Railway → Variables</i>	■
8	Update CORS_ALLOWED_ORIGINS to include https://leaveapp.in	<i>Railway → Variables</i>	■
9	Redeploy both frontend and backend after env var changes	<i>Vercel + Railway</i>	■

8. CI/CD — GitHub Actions

Auto-test + auto-deploy on every push to main

Railway and Vercel already auto-deploy when you push to GitHub. GitHub Actions adds automated **testing before deploy** — so broken code never reaches production.

Step A — Add GitHub Secrets

These are secret values the workflow uses. Never put them in code files.

#	Task	Where	Do ne
1	Go to GitHub repo → Settings → Secrets and variables → Actions	<i>GitHub</i>	■
2	Click New repository secret → add DJANGO_SECRET_KEY	<i>GitHub Secrets</i>	■
3	Add DB_NAME, DB_USER, DB_PASSWORD (use test values matching your CI database)	<i>GitHub Secrets</i>	■
4	Add RAILWAY_TOKEN if you want GitHub Actions to trigger Railway deploy directly (get token from Railway → Account Settings → Tokens)	<i>GitHub Secrets</i>	■

Step B — Create Workflow File

#	Task	Where	Do ne
1	Create folder in repo: .github/workflows/	<i>Your local machine / GitHub</i>	■
2	Create file: .github/workflows/deploy.yml	<i>Repo</i>	■
3	Workflow must do these things in order — see workflow structure table below	<i>deploy.yml file</i>	■

Workflow Step	What It Does	Required?
Trigger on push to main	Workflow runs automatically when code pushed	Yes
Checkout code	GitHub Action clones your repo into the runner	Yes
Set up Python 3.11	Install correct Python version	Yes
Install requirements	pip install -r backend/requirements.txt	Yes
Run Django tests	python manage.py test — fails workflow if tests fail	Yes
Set up Node.js	Install Node for frontend build check	Optional
npm install + build	Verify React builds without errors	Optional
Notify Railway to deploy	Railway re-pulls from GitHub and redeploys	Auto (no step needed)
Post status to Slack/email	Alert team on success or failure	Optional

✓ Railway and Vercel listen to GitHub pushes automatically. GitHub Actions only needs to run your tests — deployment happens on its own.

■ If tests fail in GitHub Actions, the push is marked red. Railway and Vercel still deploy (they don't wait for Actions). To block deploy on test failure, disable auto-deploy in Railway and trigger it manually from the workflow using Railway CLI.

9. End-to-End Verification

Do this after everything is deployed

#	Task	Where	Done
1	Open your domain (or Vercel URL) in browser — login page loads	Browser	■
2	Login as HR Admin — dashboard shows correctly	Browser	■
3	Create a department and assign a manager	Browser — HR Admin	■
4	Create an employee account	Browser — HR Admin	■
5	Login as employee — check leave balance visible	Browser — Employee	■
6	Apply for leave as employee	Browser — Employee	■
7	Login as manager — approve the leave request	Browser — Manager	■
8	Login as employee again — verify status updated to Approved, balance deducted	Browser — Employee	■
9	Login as HR Admin — check HR dashboard shows the leave	Browser — HR Admin	■
10	Export CSV from HR dashboard — file downloads correctly	Browser — HR Admin	■
11	Push a small code change to GitHub → verify CI runs → verify auto-redeploy happens	GitHub → Railway/Vercel	■
12	Check HTTPS lock icon in browser — SSL working on all URLs	Browser	■

10. Common Problems & Fixes

Problem	Likely Cause	Fix
Login works locally, fails on prod	CORS not configured for Vercel URL	Add Vercel URL to CORS_ALLOWED_ORIGINS in backend env
API returns 500 error	DEBUG=False hides error — check logs	Railway/Render → Logs tab → read the actual Python error
Frontend shows blank page	React build failed or REACT_APP_API_URL wrong	Check Vercel build logs. Verify env var has no trailing slash
Database connection error	DATABASE_URL not set or wrong	Railway auto-sets it if PostgreSQL plugin added to same project
migrations not run	Forgot to run in production shell	Railway/Render → Shell → python manage.py migrate
Domain not working after DNS add	DNS not propagated yet	Wait 30 min. Check at dnschecker.org
GitHub Actions test fails	Test uses local DB that doesn't exist in prod	Add sqlite DB config in settings for test environment or use GitHub Actions postgresql service
Vercel deploy fails	Build error in React code	Read Vercel deploy logs — error shown clearly with line number
SSL not working on custom domain	DNS not pointing correctly	Verify CNAME records match exactly what platform gave you